

Exercise 1:

// Sum of first n natural numbers (not in the best possible way though)

```
#include <stdio.h>

int
main(int argc, char **argv)
{
    int i;
    int sum;
    int n = 10;

    sum = 0;
    for (i = 1; i <= n; i++)
        sum += i;

    printf("Sum of first %d natural numbers is: %d\n", n, sum);

    return 0;
}
```

- Identify the basic blocks. Draw a Control Flow Graph (CFG) using these basic blocks.
- Calculate McCabe's complexity from the CFG so obtained .

Solution 1: McCabe's complexity = 2 (E=9, N=9 based on CFG)

Exercise 2:

Find the cyclomatic complexity of Euclid's, **gcd**, function in following C program. If possible, find test cases that allow you to test each *independent path* of the **gcd** function. How many test cases do you actually need to cover all source lines of the **gcd** function?

```
#include <stdio.h>

int gcd(int i, int j){
    int small, large, remainder;

    if (i <= j)
        small = i;
    else
        small = j;

    if (i <= j)
        large = j;
    else
        large = i;

    while (small > 0){
        remainder = large % small;
        large = small;
        small = remainder;
    }
    return large;
}

int main(void) {
    int i, j;

    printf("Enter two positive integers: ");
    scanf("%d %d", &i, &j);
}
```

```

    printf("GCD of %d and %d is %d\n\n", i, j, gcd(i,j));
    return 0;
}

```

Solution 2:

First draw the CFG.

The cyclomatic complexity of the **gcd** function is **4**. If main() function is also considered, the total cyclomatic complexity will be **5**.

In general, **the cyclomatic complexity is an upper limit of the number of necessary test cases**. In the **gcd** function, it is possible to cover all source lines with even *two* test cases.

Notes on Test Coverage

Statement Coverage:

In this the test case is executed in such a way that **every statement of the code is executed at least once**.

Branch/Decision Coverage:

Test coverage criteria requires enough test cases such that each condition in a decision takes on all possible outcomes at least once, and each point of entry to a program or subroutine is invoked at least once. That is, **every branch (decision) taken each way, true and false**. It helps in validating all the branches in the code making sure that no branch leads to abnormal behavior of the application.

Path Coverage:

In this the test case is executed in such a way that every path is executed at least once. All possible control paths taken, including all loop paths taken zero, once, and multiple (ideally, maximum) items in path coverage technique, the test cases are prepared based on the logical complexity measure of a procedural design. In this type of testing every statement in the program is guaranteed to be executed at least one time. Flow Graph, Cyclomatic Complexity and Graph Metrics are used to arrive at basis path .

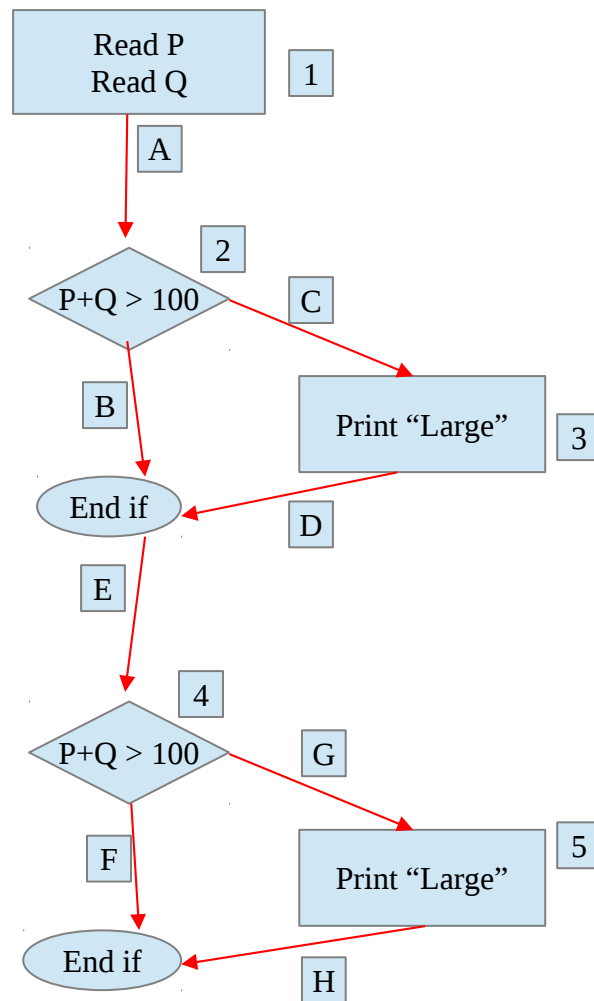
For all test coverage exercises, start with drawing the control flow graph (CFG).

Exercise 3: Calculate Statement Coverage, Branch Coverage and Path Coverage for the following code segment.

```

Read P
Read Q
IF P+Q > 100 THEN
Print "Large"
ENDIF
If P > 50 THEN
Print "P Large"
ENDIF

```



CFG for code segment given in Exercise 3

Solution 3:

Statement Coverage (SC):

To calculate Statement Coverage, **find out the shortest number of paths following which all the nodes will be covered.** Here by traversing through path **1A-2C-3D-E-4G-5H** all the nodes are covered. So by traveling through only one path all the nodes 12345 are covered, so the **Statement coverage (SC) in this case is 1.**

Branch Coverage (BC):

To calculate Branch Coverage, **find out the minimum number of paths which will ensure covering of all the edges.** In this case there is no single path which will ensure coverage of all the edges at one go. By following paths **1A-2C-3D-E-4G-5H**, maximum numbers of edges (A, C, D, E, G and H) are covered **but edges B and F are left.** To covers these edges we can follow **1A-2B-E-4F**. By the combining the above two paths we can ensure of traveling through all the paths. Hence **Branch Coverage (BC) is 2.** The aim is to cover all possible true/false decisions.

Path Coverage (PC):

Path Coverage ensures covering of all the paths from start to end. All possible paths are -

1A-2B-E-4F, 1A-2B-E-4G-5H, 1A-2C-3D-E-4G-5H, 1A-2C-3D-E-4F

So path coverage (PC) is 4.

Thus for the above example SC=1, BC=2 and PC=4.